Wrangling Telemetry at Scale: A Guide to Self-Hosted Observability

Colin Douch, SRE @ DuckDuckGo

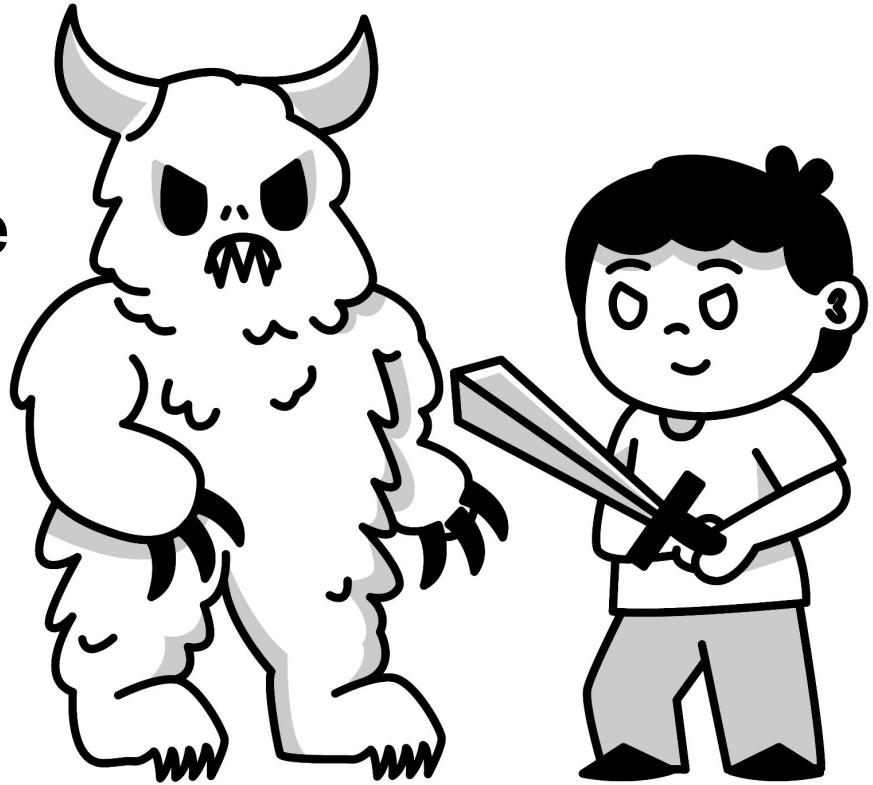
Do you ever feel the
complexity demon
creeping up on you?



We've all developed hacks to try and escape it, but it's ***always*** there



But sometimes you have to confront it, and when you do, you need ***Observability***



Should I run my own Observability stack?



No*

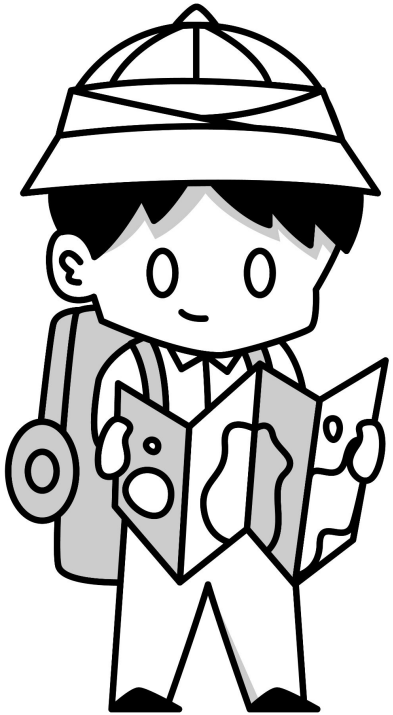
****

*At least not until you've exhausted each and every other option

But… Sometimes it becomes necessary

- Billing woes are *real*
- Auditors might start asking tricky questions about why third parties have so much data
- Maybe you've asked for a feature, and your Observability provider has fobbed you off





- What does self hosted observability look like?
- What are the trade-offs?
- How can we weave our tooling together into something that gives us real observability

Hi! I’m Colin

- Formerly formerly a Founding SRE @ Xero
- Formerly founded and led the Observability Team @ Cloudflare
- Currently at DuckDuckGo working on privacy preserving telemetry at scale
- On the wrong side of the world



Be prepared for what that actually entails



- *At least* an extra 2-3 engineers full time
- Spending a *huge* amount of **money**
- Spending more **time** trawling Github Issues pages than you have your entire life
- Putting enough **effort** into it to make you rethink all your life choices

But what does that look like?

As much as O11y companies want to pretend otherwise: most people are still looking at the “three pillars”: metrics, logs, and traces

But we should also talk about ways we can stitch those pillars together into a coherent canvas



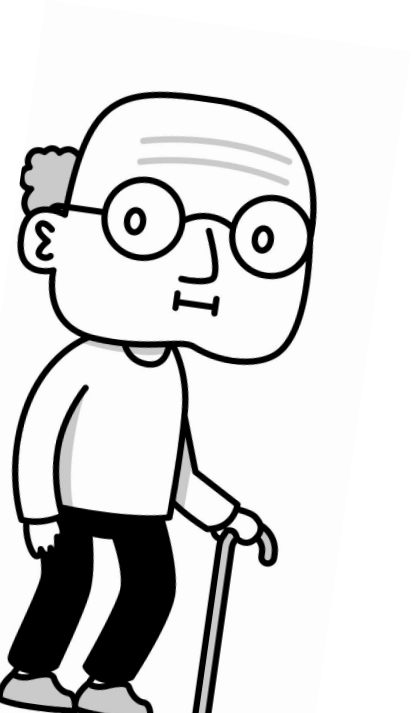
“What should I use to store my metrics?”

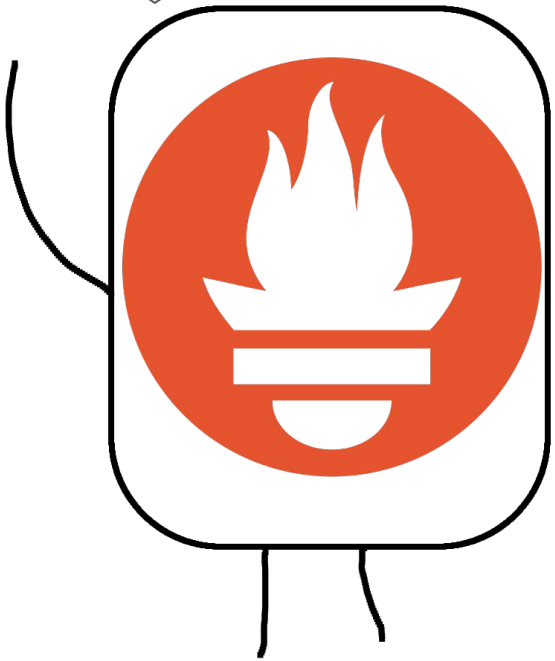

Prometheus….
Obviously


...
Maybe
VictoriaMetrics?

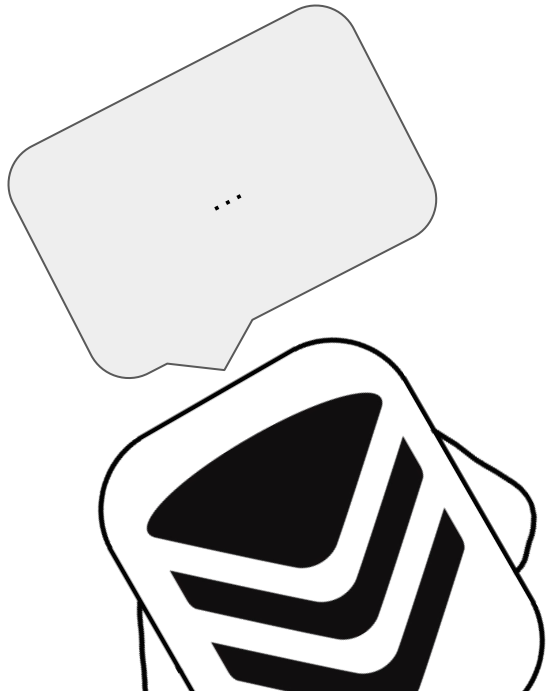
How about Graphite or rrd?

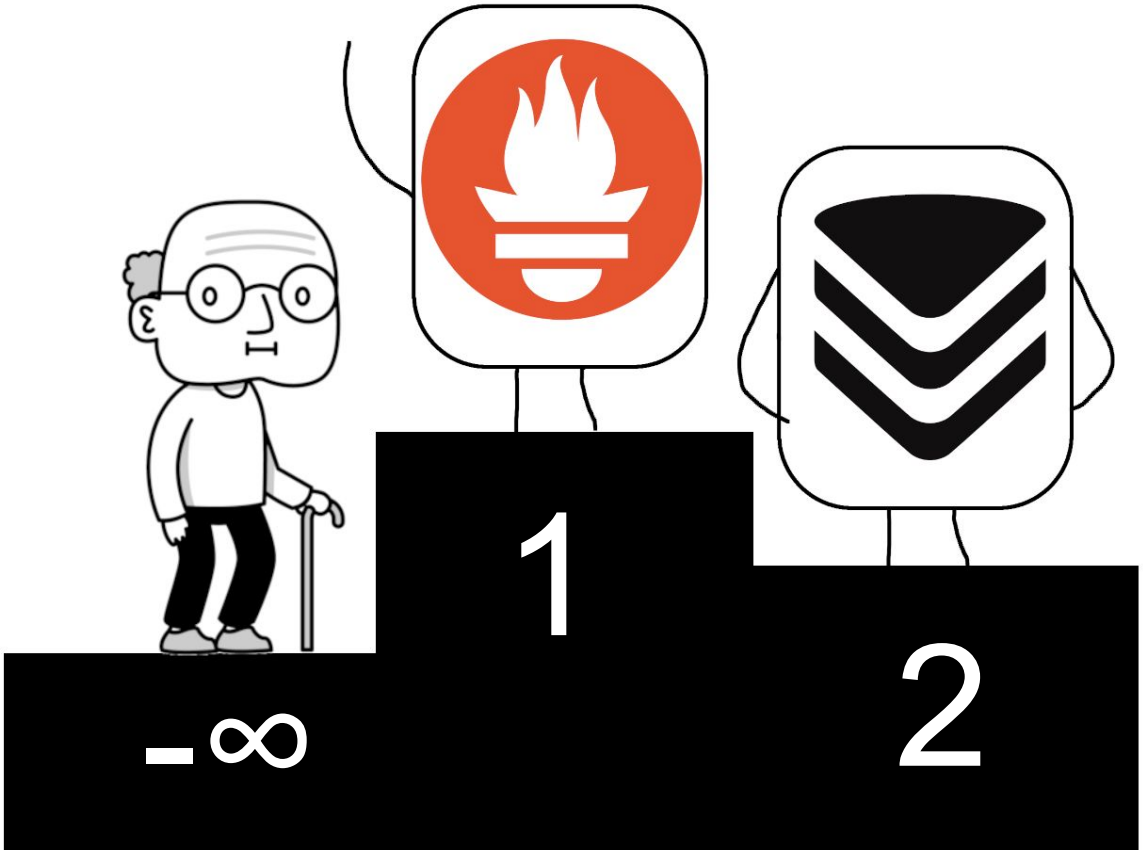
…





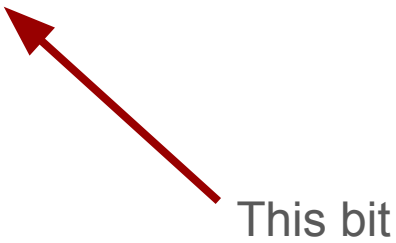
…

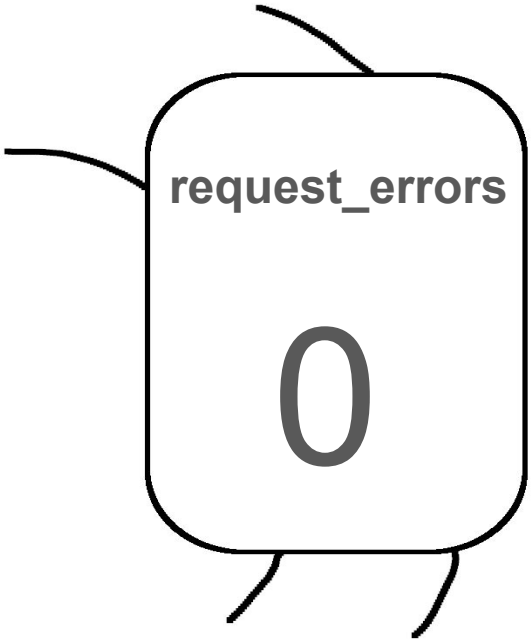



1
2
-∞

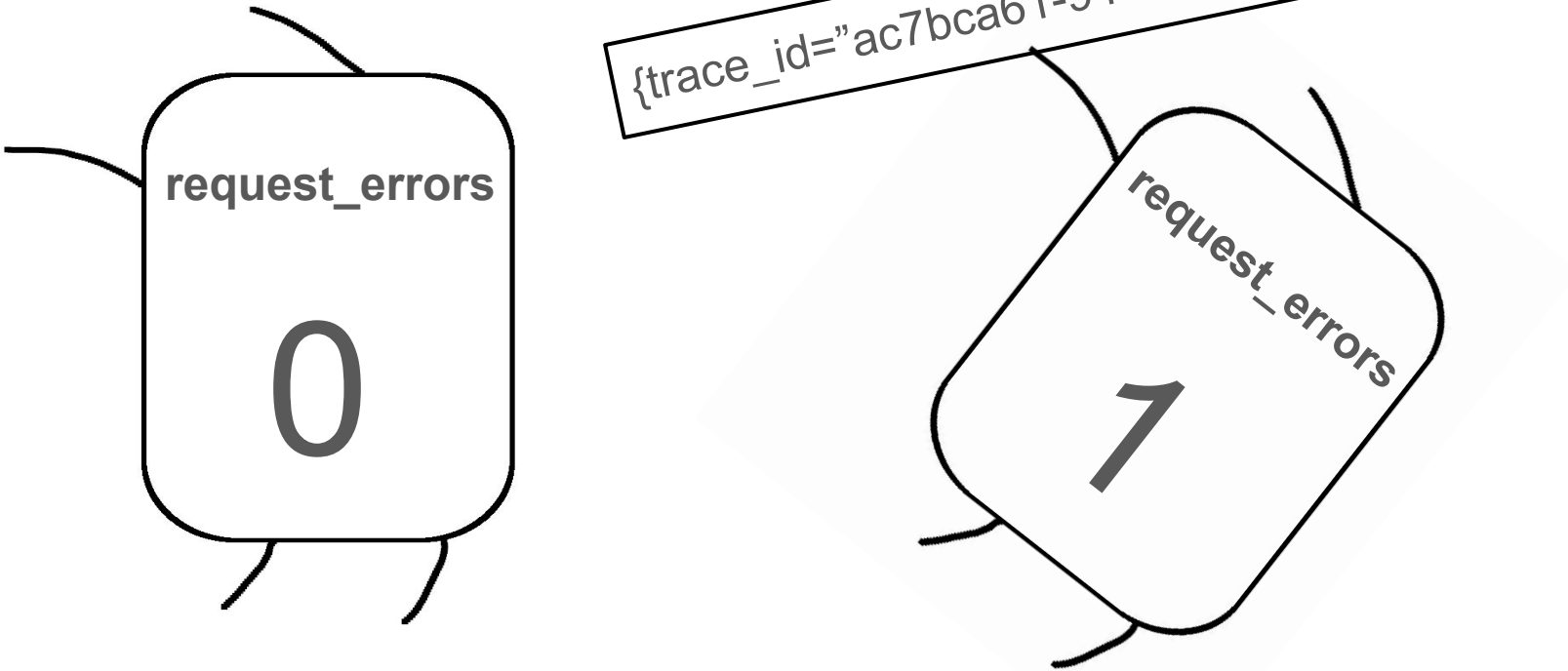
```
--enable-feature=exemplar-storage
```

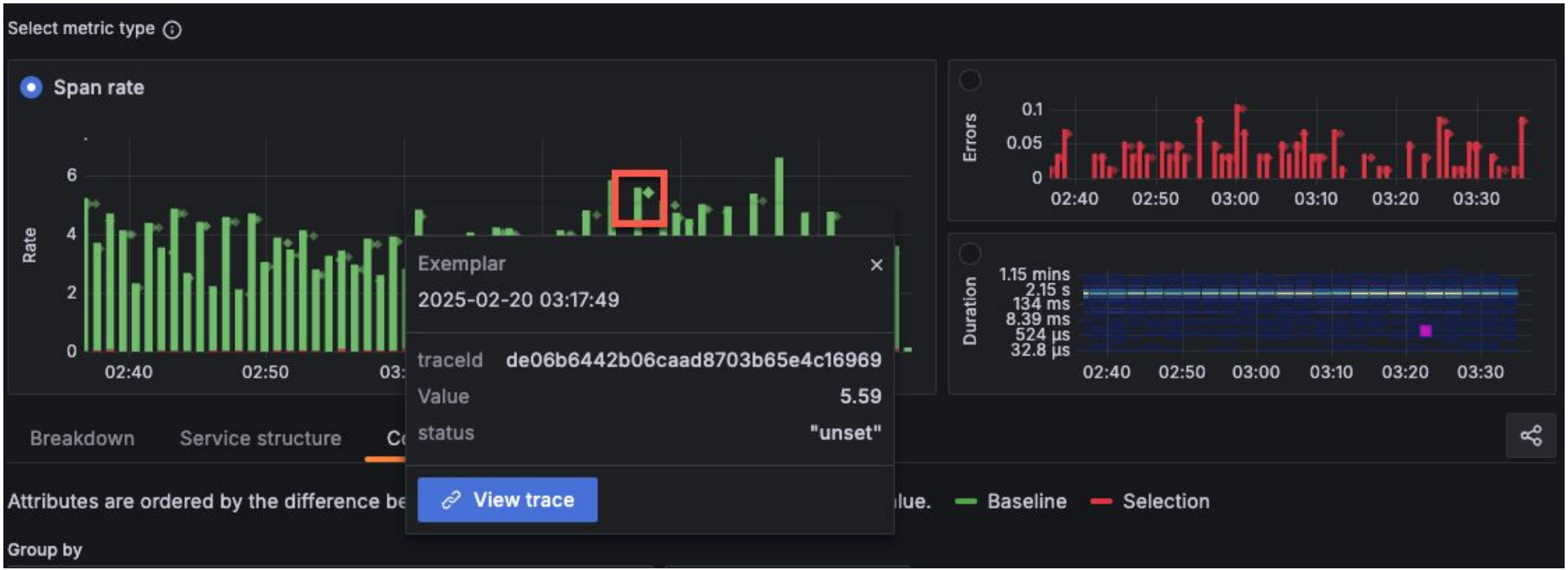
```
http_request_errors_total{handler="/api/search",status="500"} 1.0 #
{traceID="4bf92f3577b34da6a3ce929d0e0e4736"} 1.0 1.7725568641686323e+09
```




request_errors
0

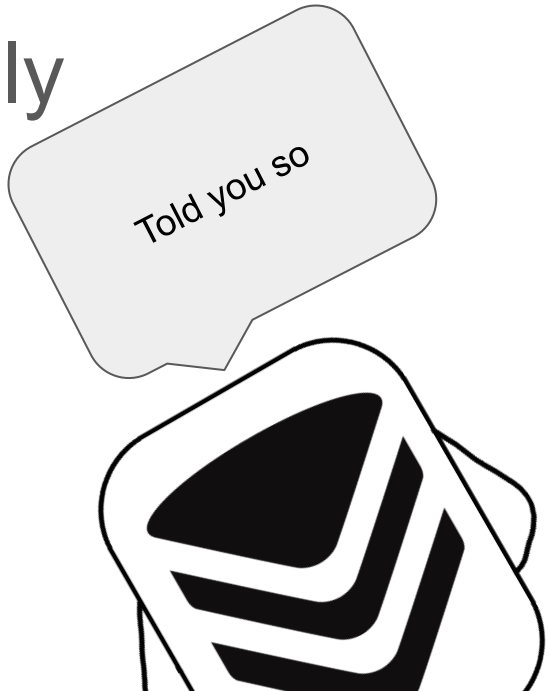

request_errors
1

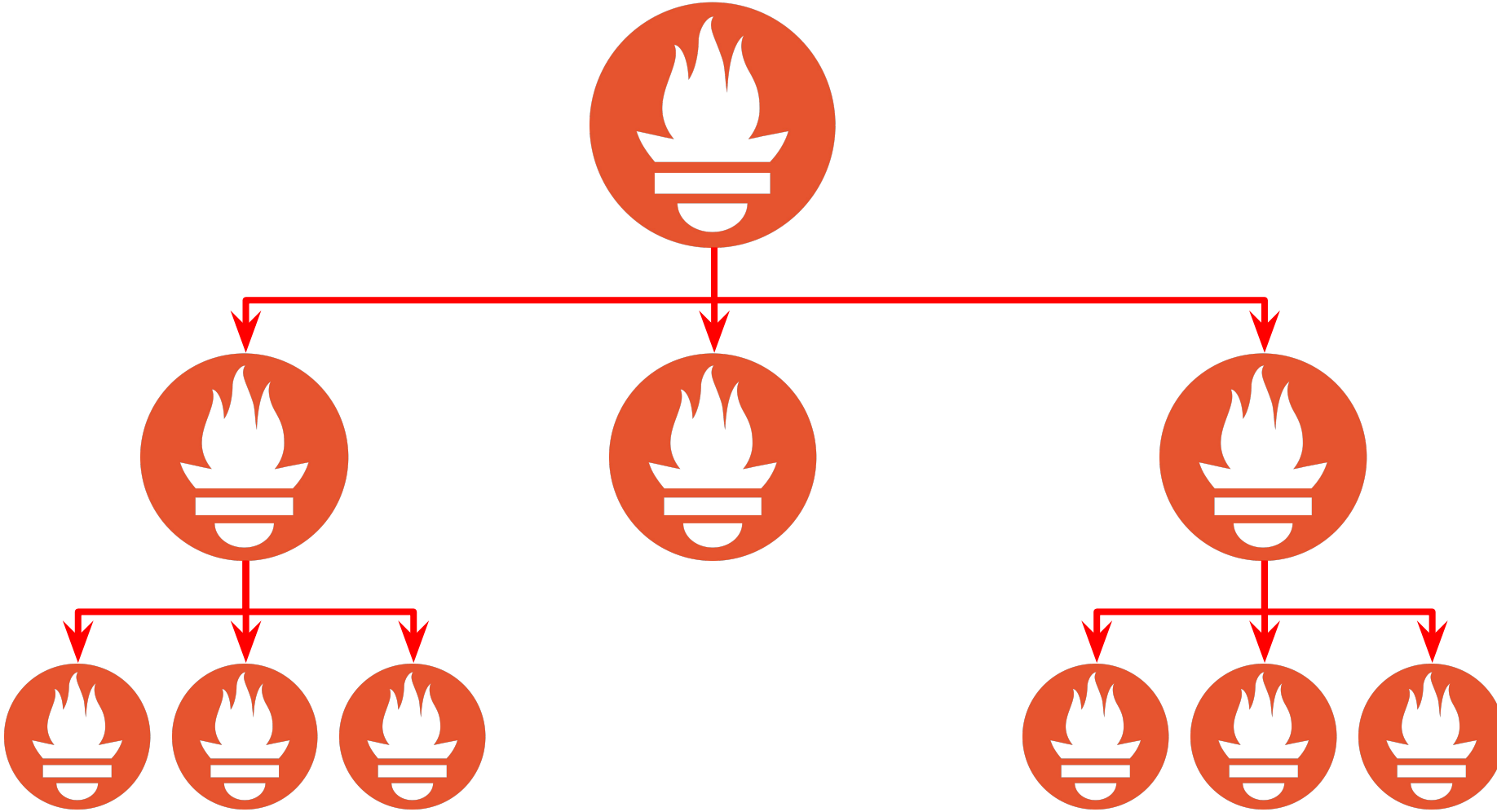

request_errors
0
{trace_id=”ac7bca61-944a-4518-9a02”}
request_errors
1


Select metric type
Span rate
Rate
6
4
2
0
02:40
02:50
Errors
0.1
0.05
0
02:40
02:50
03:00
03:10
03:20
03:30
Duration
1.15 mins
2.15 s
134 ms
8.39 ms
524 µs
32.8 µs
02:40
02:50
03:00
03:10
03:20
03:30
Exemplar
2025-02-20 03:17:49
traceId de06b6442b06caad8703b65e4c16969
Value 5.59
status "unset"
View trace
Breakdown
Service structure
Attributes are ordered by the difference be
lue.
Baseline
Selection
Group by

Other things scale horizontally

Prometheus just… doesn’t



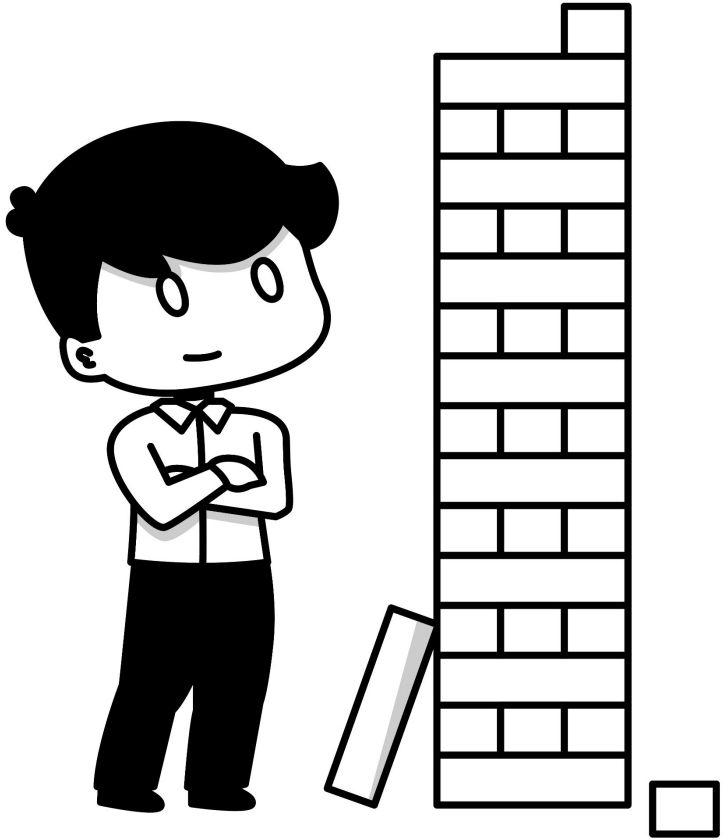


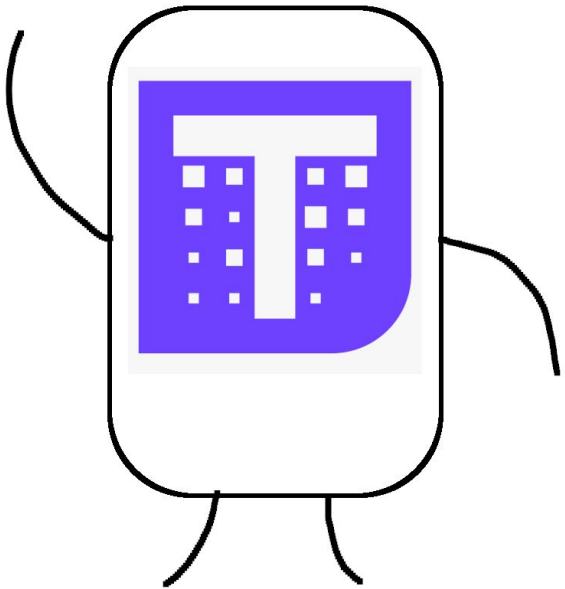


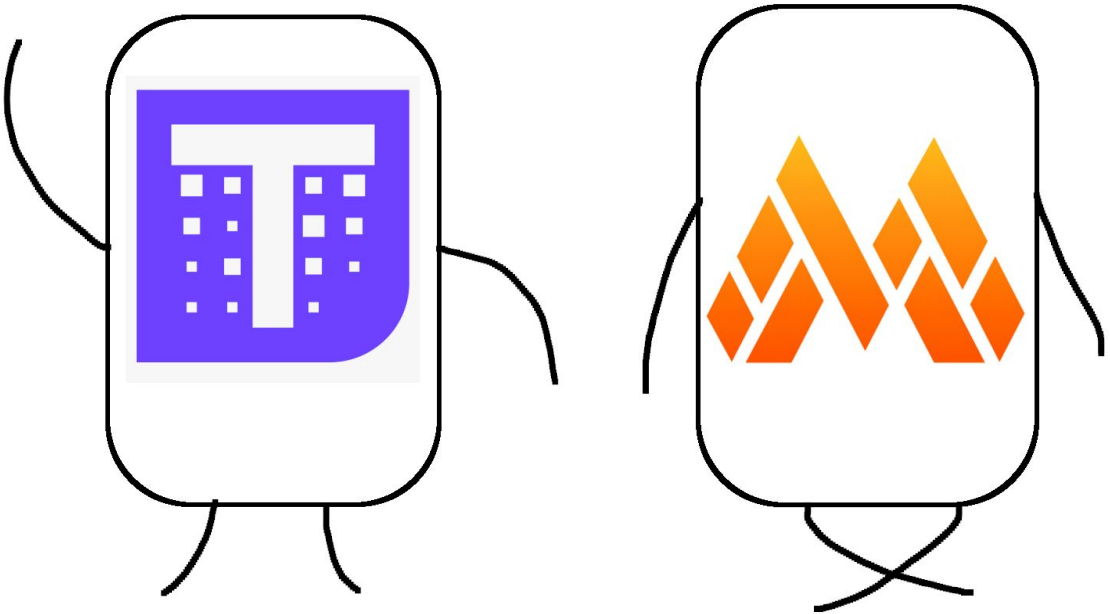
So ***how do you scale Prometheus?***

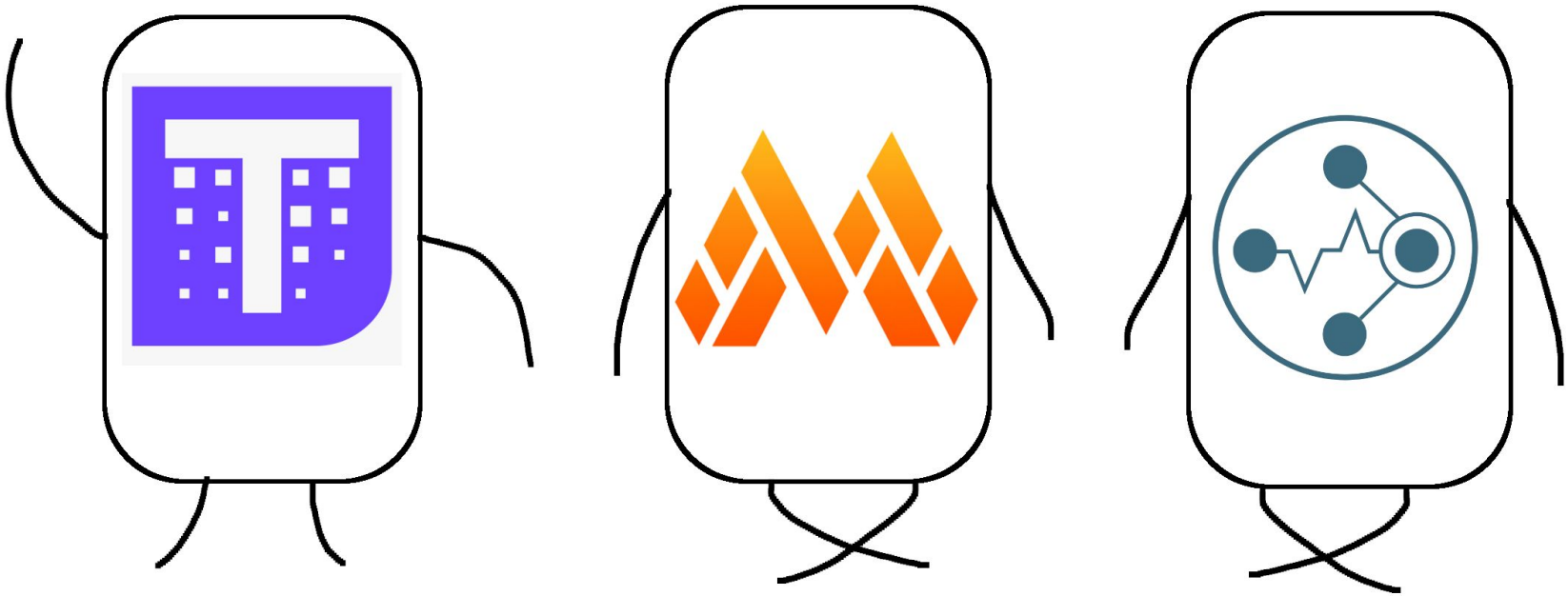
For active timeseries in the Head Block, there's not much you can do except vertical scaling and sharding.

As a rough rule of thumb: 4KB of RAM per timeseries for your capacity planning

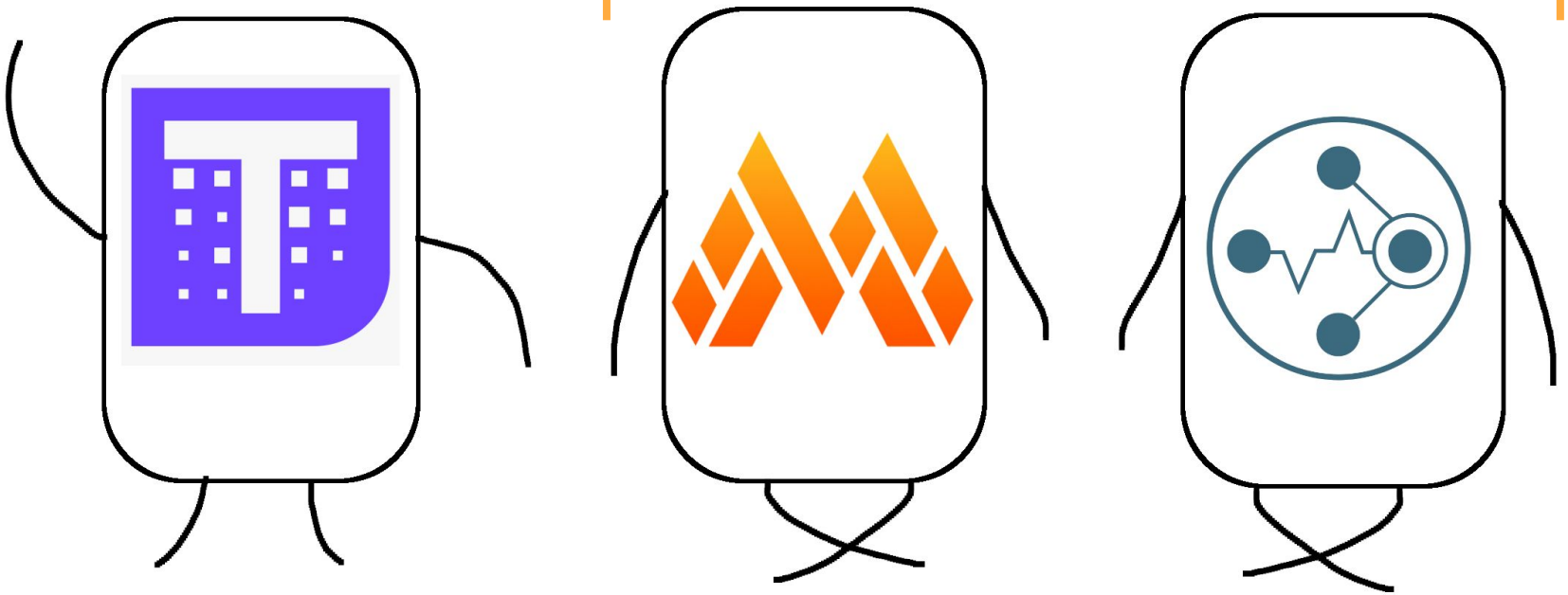




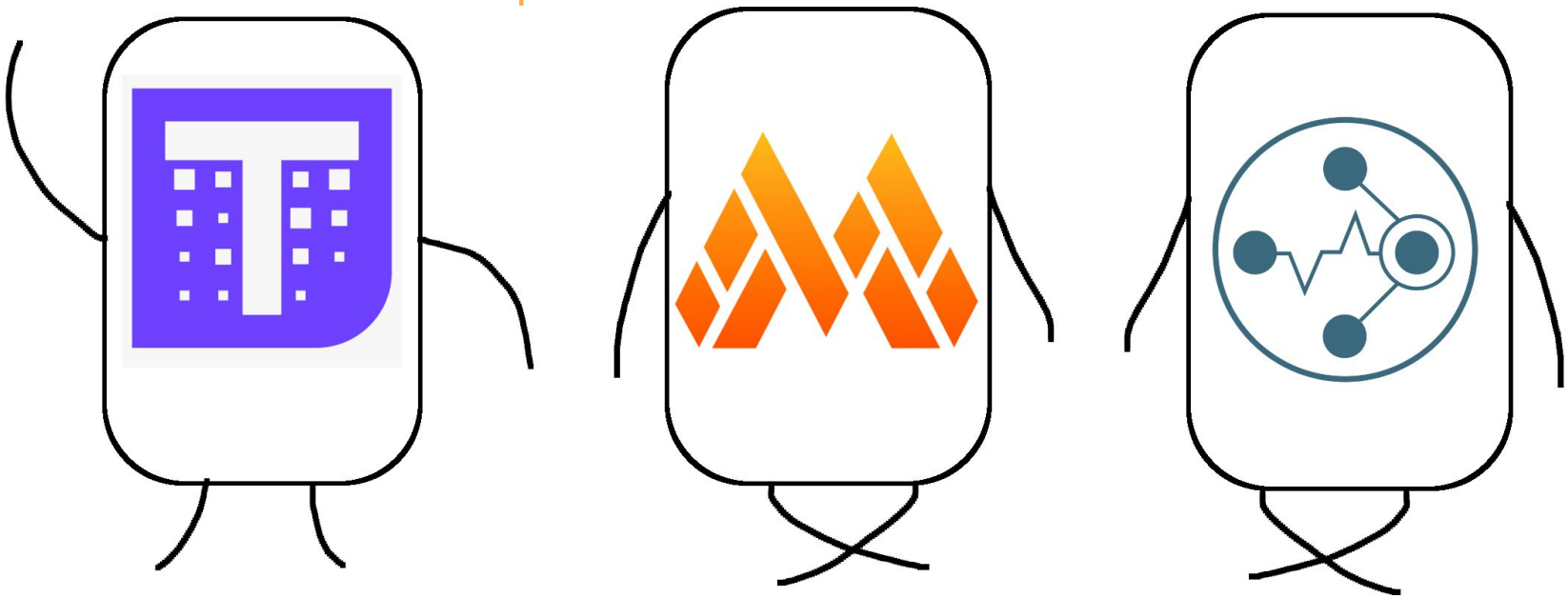




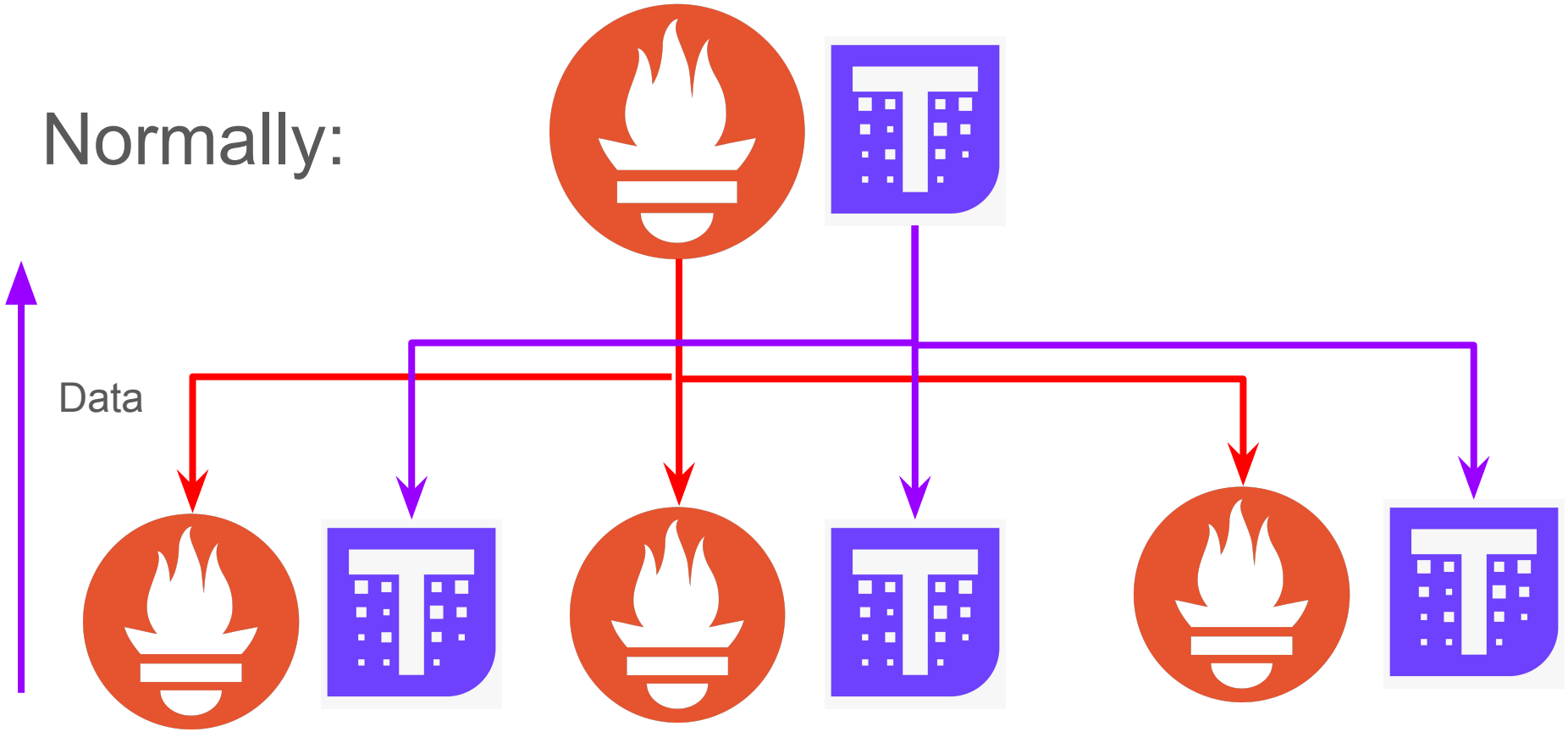
Remote write



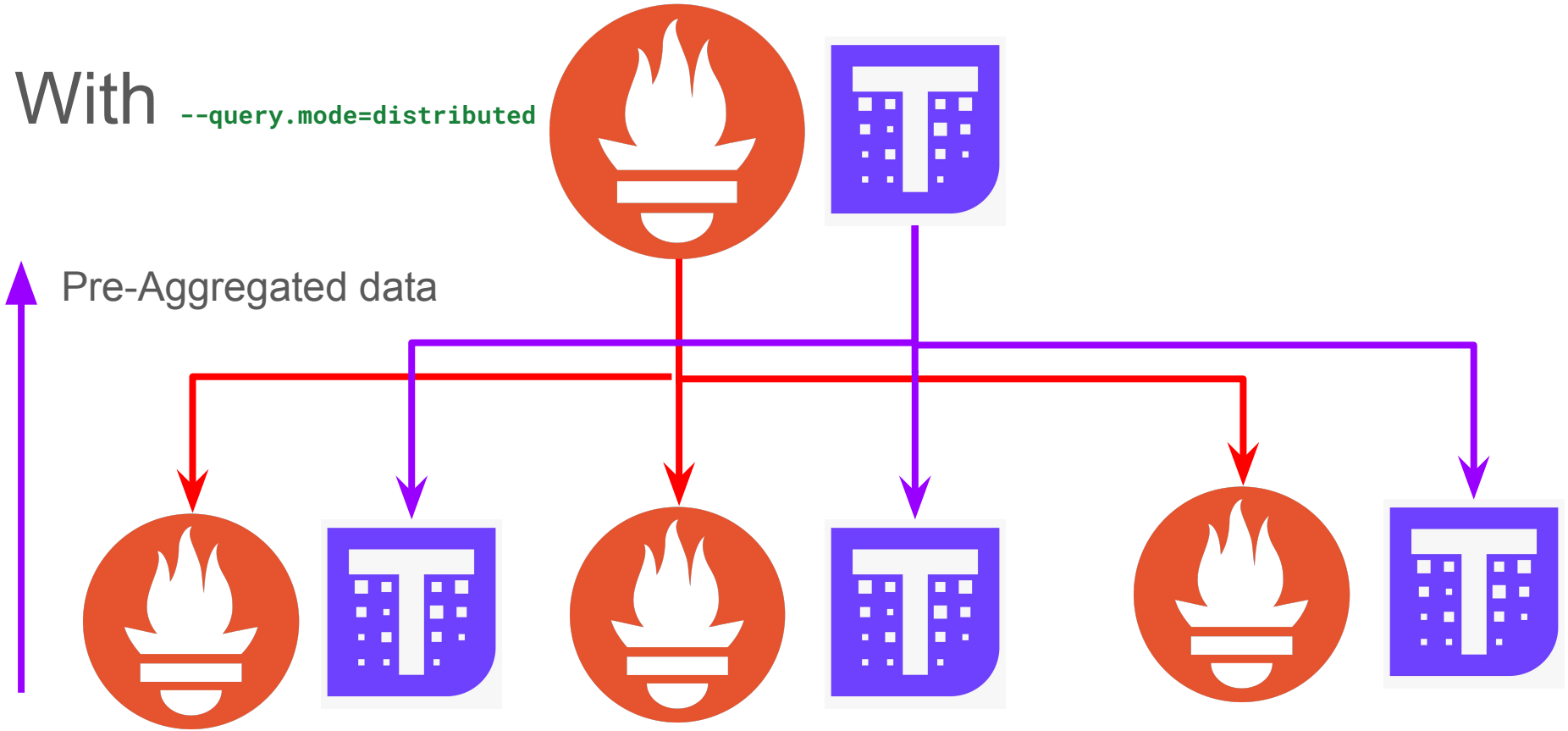
Blocks



Normally:



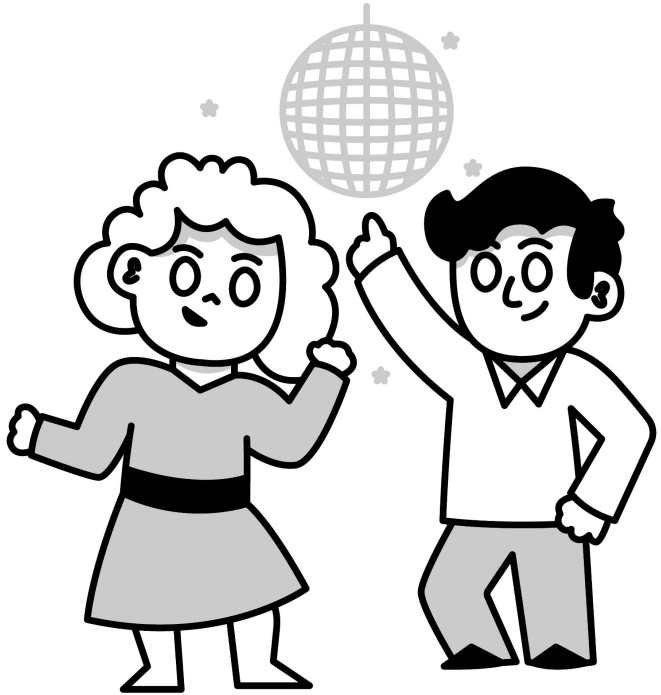
With `--query.mode=distributed`



https://github.com/thanos-io/thanos-parquet-gateway



Let’s talk about logs



```
// Authenticate
log.Debug("checking user session", "userID", userID)
if userID == "" {
  log.Warn("unauthenticated search request")
  return errors.New("unauthenticated")
}
log.Debug("session valid", "userID", userID)

// Parse and normalise the query
log.Info("parsing query", "raw", query)
tokens := tokenise(query)
log.Info("query parsed", "tokens", len(tokens), "normalised", strings.Join(tokens, " "))

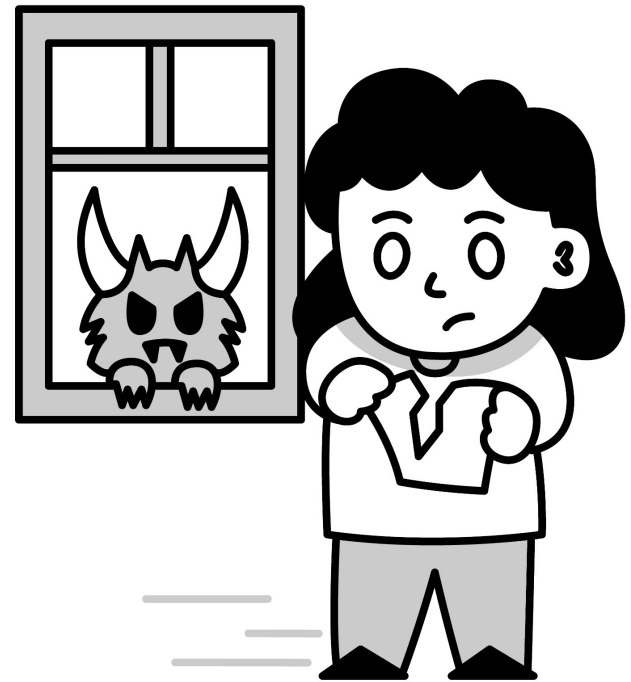
// Check query cache
log.Debug("checking query cache", "query", query)
cacheHit := rand.IntN(10) > 6
log.Info("cache lookup complete", "hit", cacheHit)
if cacheHit {
  log.Info("serving from cache", "query", query)
  return nil
}

// Fetch candidates from index shards in parallel
log.Info("querying index shards")
```

“Logs are bad and you should feel bad!”

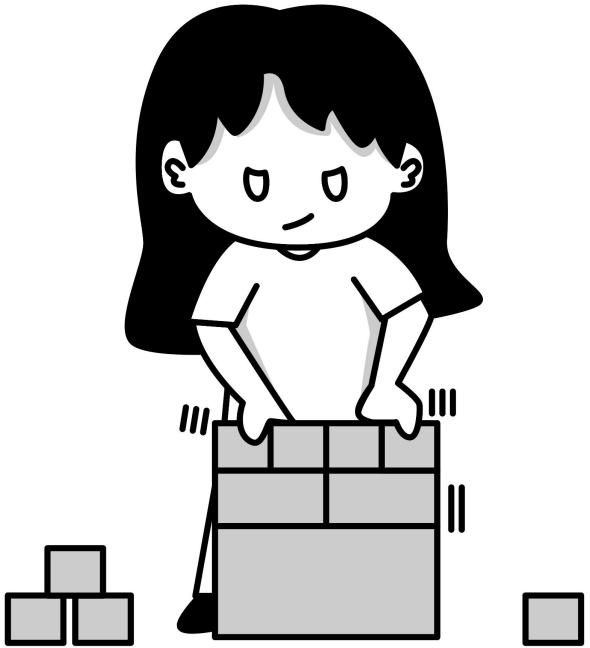
“All you need is tracing, you dolt!”

“Any one who mentions the three pillars will be stung to death by bees!” - A Honeycomb engineer, probably



Step 1: Structure your logs

- Don’t log in plain text, add some structure
- Use a database that can handle that structure
- Add a lot of context to that structure - the more the better!

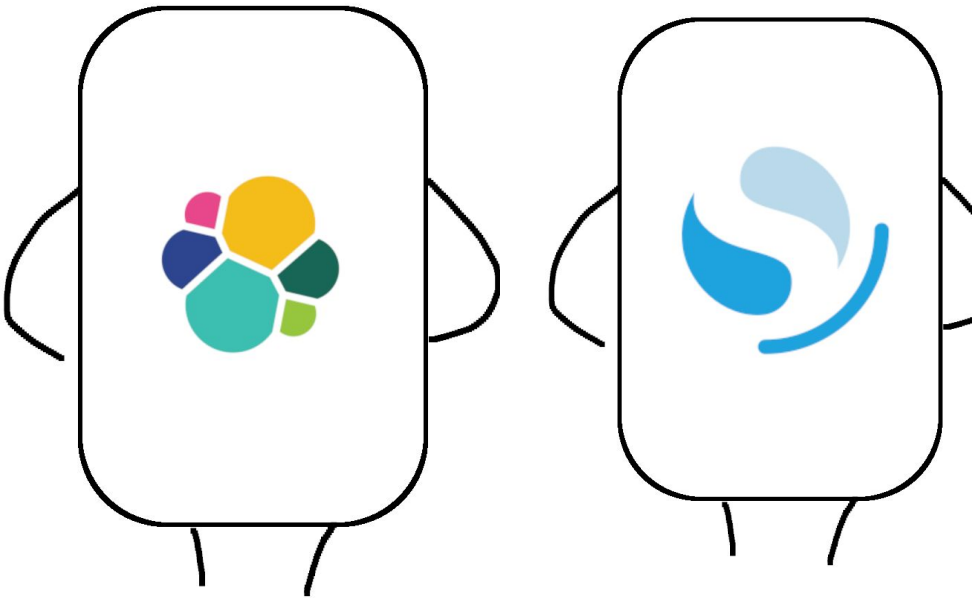


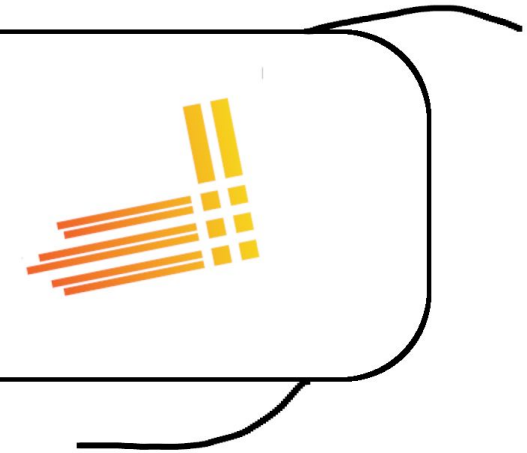
```
subscriptions getUser user-529 timeout
subscriptions getUser user-20 network error
subscriptions getUser user-148 timeout
subscriptions getUser user-181 timeout
subscriptions getUser user-998 network error
subscriptions getUser user-2070 timeout
subscriptions getUser user-909 timeout
subscriptions getUser user-289 invalid input
subscriptions getUser user-828 network error
subscriptions getUser user-202 invalid input
```

```
subscriptions getUser user-20 network error
subscriptions getUser user-2070 timeout
subscriptions getUser user-202 invalid input
```

Step 2: Store your logs in a decent database

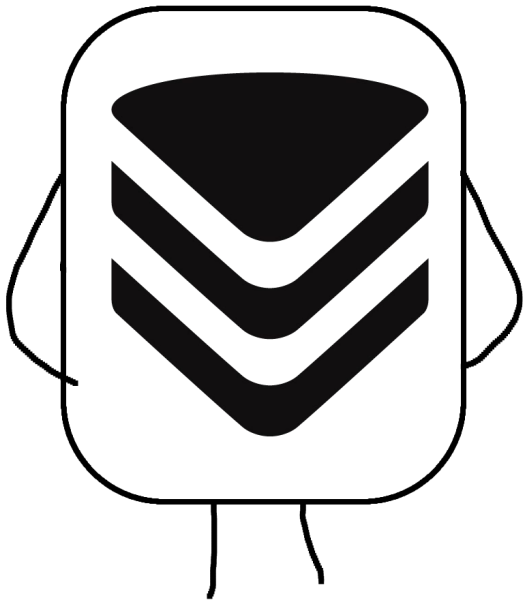
- You need a storage engine that indexes your structured fields
- You **need** support for dynamic schemas





- **Loki** was the first *real, dedicated* log storage, rather than just a search engine
- It based a lot of its ideas on Prometheus, which is really *interesting* - it indexes ***specific labelsets*** from your logs
- But that means you have to define a ***schema upfront***

- VictoriaLogs came later, but is an ***actual proper columnar database***
- No need to define your fields up front, just ingest everything
- Now that's in the spirit of ***proper observability***





- You ***could*** cut out the middleware and just ingest into a columnar database
- Cloudflare actually got quite far just inserting into Clickhouse and using things like ***Quesma*** for querying
- ***Don’t do this*** unless you are already running one

Sprinkling in logs leads to a soup of unusable data that makes it nigh on ***impossible to solve problems***



Step 3: Canonical Logging

```
// Authenticate
log.Debug("checking user session", "userID", userID)
if userID == "" {
  log.Warn("unauthenticated search request")
  return errors.New("unauthenticated")
}
log.Debug("session valid", "userID", userID)

// Parse and normalise the query
log.Info("parsing query", "raw", query)
tokens := tokenise(query)
log.Info("query parsed", "tokens", len(tokens), "normalised", strings.Join(tokens, " "))

// Check query cache
log.Debug("checking query cache", "query", query)
cacheHit := rand.IntN(10) > 6
log.Info("cache lookup complete", "hit", cacheHit)
if cacheHit {
  log.Info("serving from cache", "query", query)
  return nil
}

// Fetch candidates from index shards in parallel
log.Info("querying index shards")
```

```
func handleSearch(ctx context.Context, log *slog.Logger, query string) {
  start := time.Now()
  fields := []any{"query", query}
  var err error
  defer func() {
    fields = append(fields, "durationMs", time.Since(start).Milliseconds())
    if err != nil {
      log.ErrorContext(ctx, "search", append(fields, "error", err.Error())...)
    } else {
      log.InfoContext(ctx, "search", fields...)
    }
  }()

  if results, ok := cache[query]; ok {
    fields = append(fields, "cacheHit", true, "results", len(results))
    return
  }

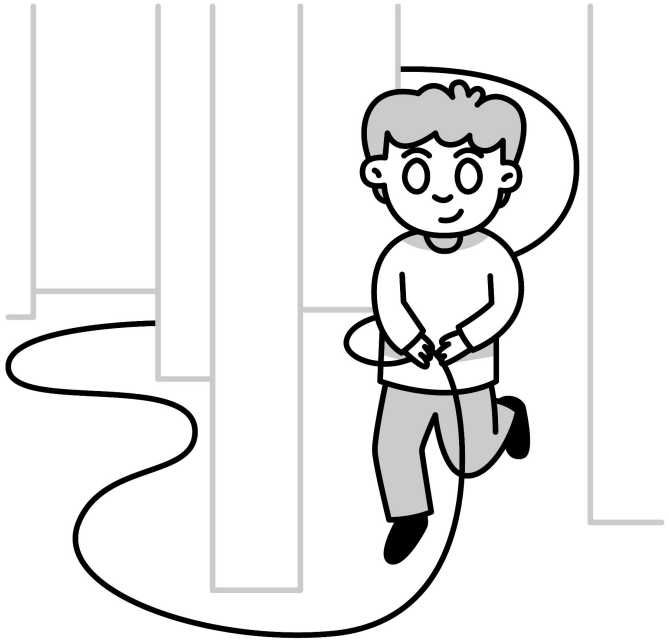
  candidates, err := queryShards(ctx, query)
  if err != nil {
    return
  }
  fields = append(fields, "candidates", len(candidates))

  results, filtered, err := rankAndFilter(ctx, candidates)
  if err != nil {
    return
  }
  fields = append(fields, "filtered", filtered, "results", len(results))
  cache[query] = results
}
```

- If we have structured logging
- And we have a proper columnar database
- And we're doing canonical logging
- That ***might be enough***



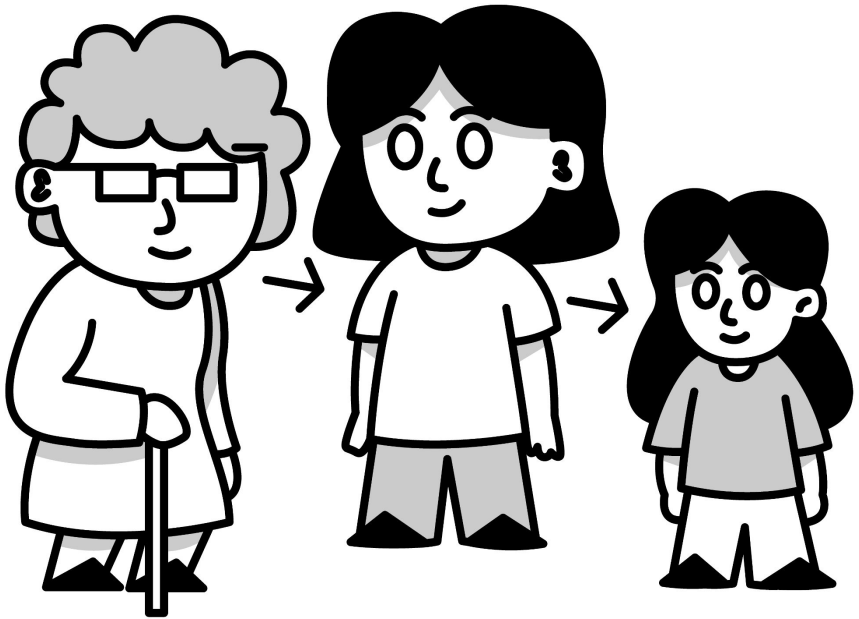
Let’s talk about tracing



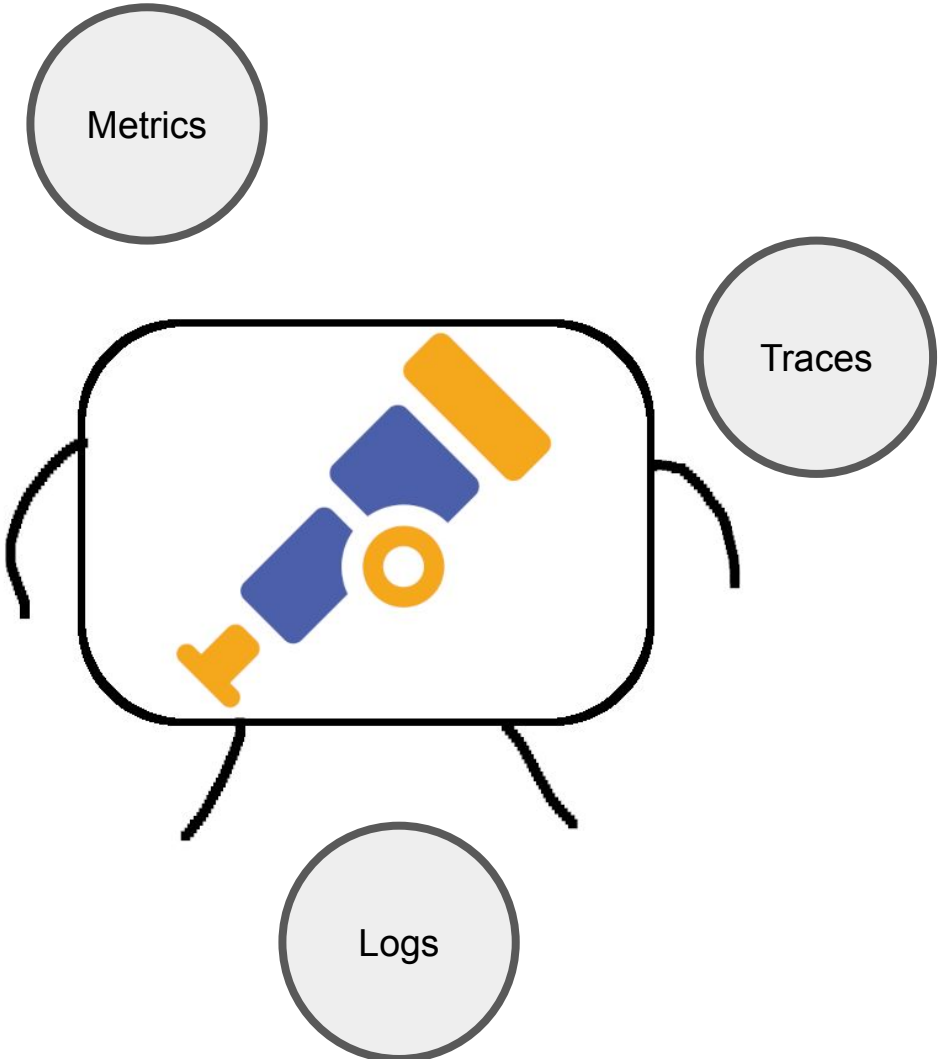


- Traces are just a ***fancy name for logs*** with some pre-agreed structure
- Ideally ***with lots of context***, like our canonical logs, but not necessarily

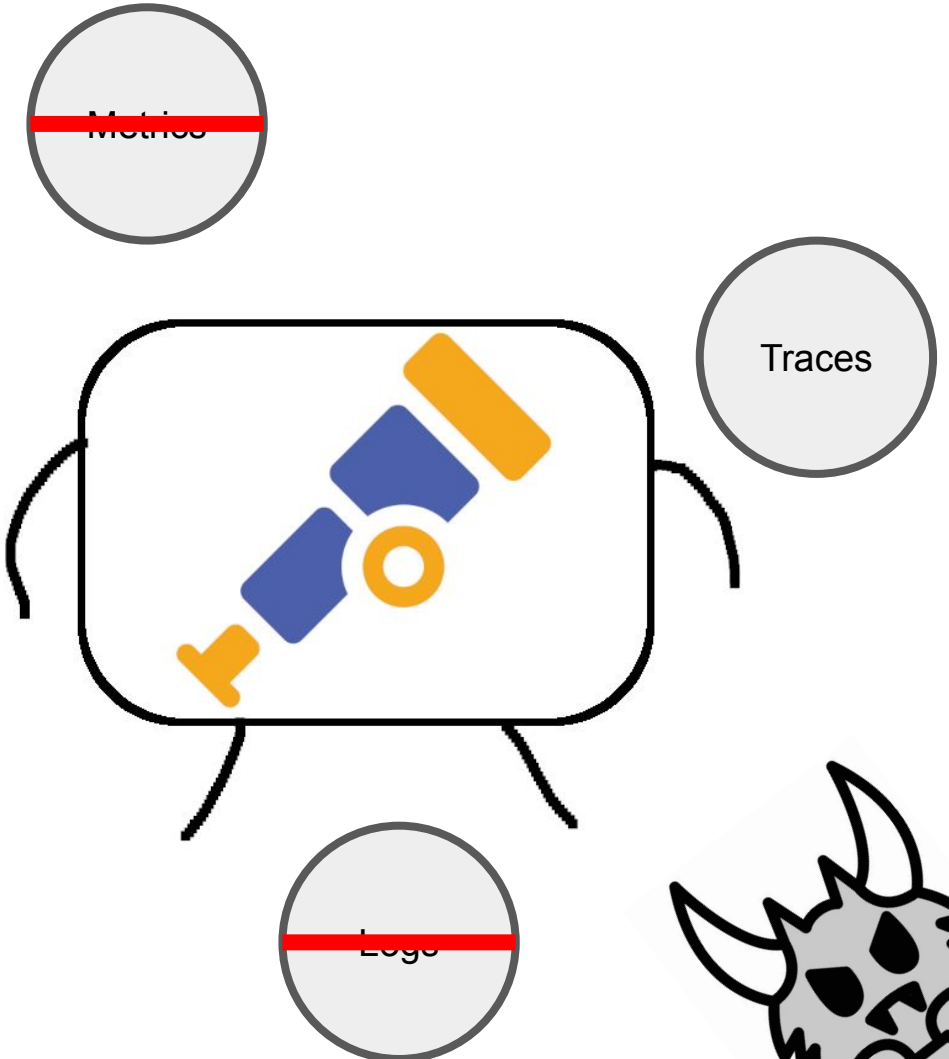
- Logs have an ***ordering problem*** across multiple processes
- Traces *solve* that problem, with ***explicit IDs*** that link together
- That's the ***only difference***



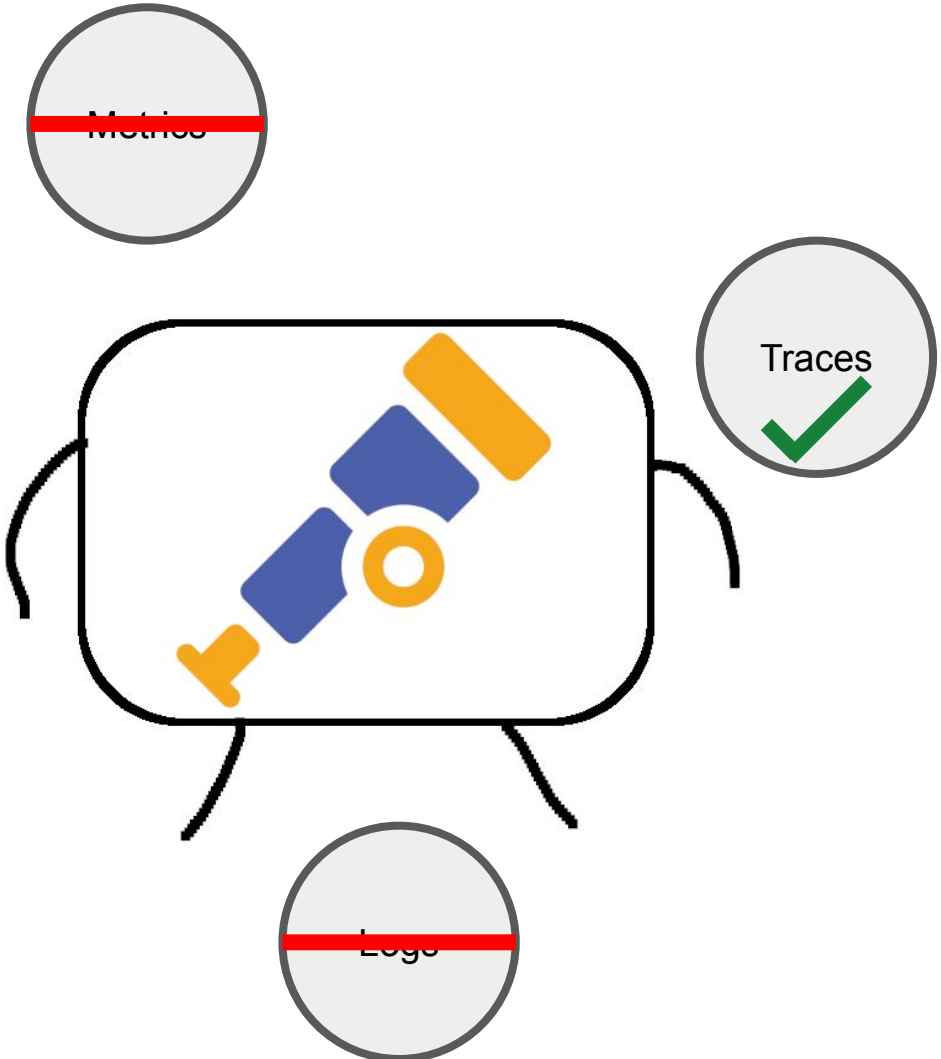
- **OpenTelemetry** is really neat
- It makes a lot of decisions for you, which helps standardise across services
- For traces, **the complexity** is worth it



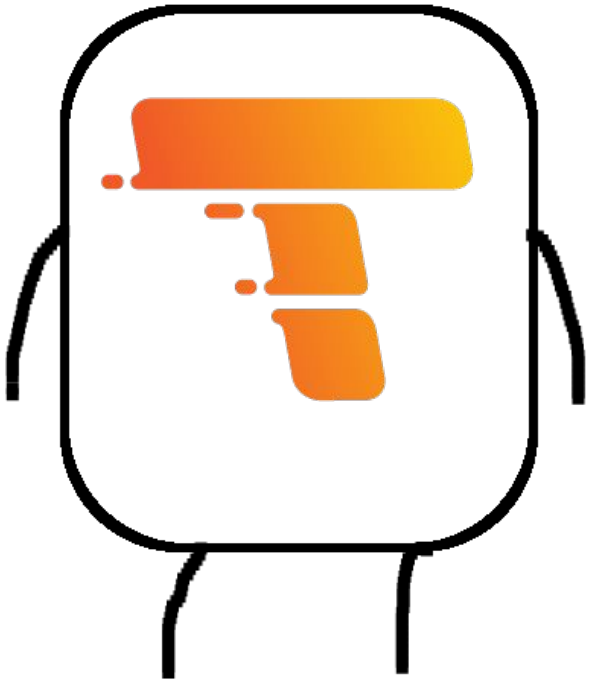
- But **don’t** use it for metrics or logs
- You can just ship Prometheus Text Exposition and JSON
- Don’t give in to the complexity demon



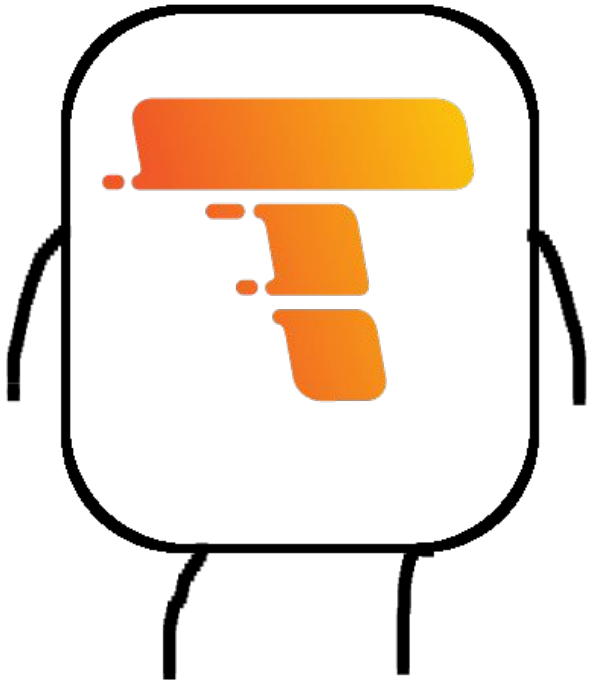
- But go right ahead for tracing
- The complexity they introduce becomes worth it
  - ***IF*** you need tracing in the first place



“Where should my traces go?”







- Uses its own storage engine that doesn’t scale very well
- But has really nice ***Observability features***
  - The ability to generate metrics from traces
  - TraceQL for filtering traces

- Storage engine agnostic, so scales nicer and is easier to run
- But lacks a lot of features
  - No proper search
  - No aggregations



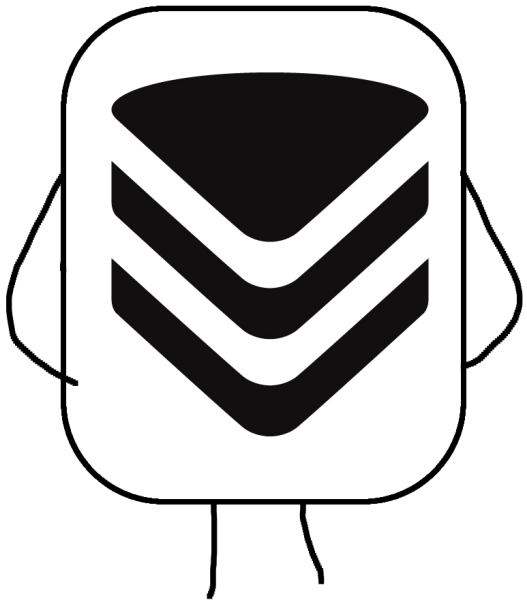


This is where Open Source Observability kind of sucks

- Tempo doesn’t scale that well, but has nice features
- Jaeger scales well, but is missing a lot of nicer Observability features

We need something better!

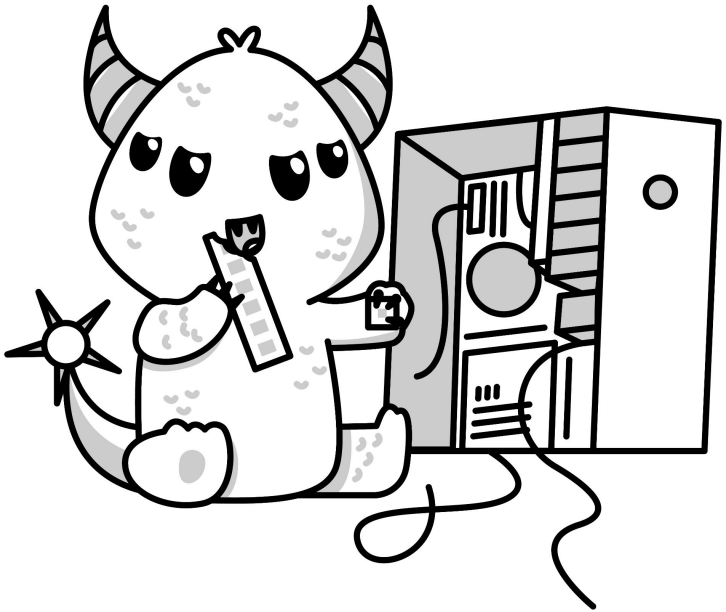
- **VictoriaTraces** *might* be the solution here, but it's still to early to tell
- Same scalable columnar DB as their logs offering
- Nice query language for filtering traces

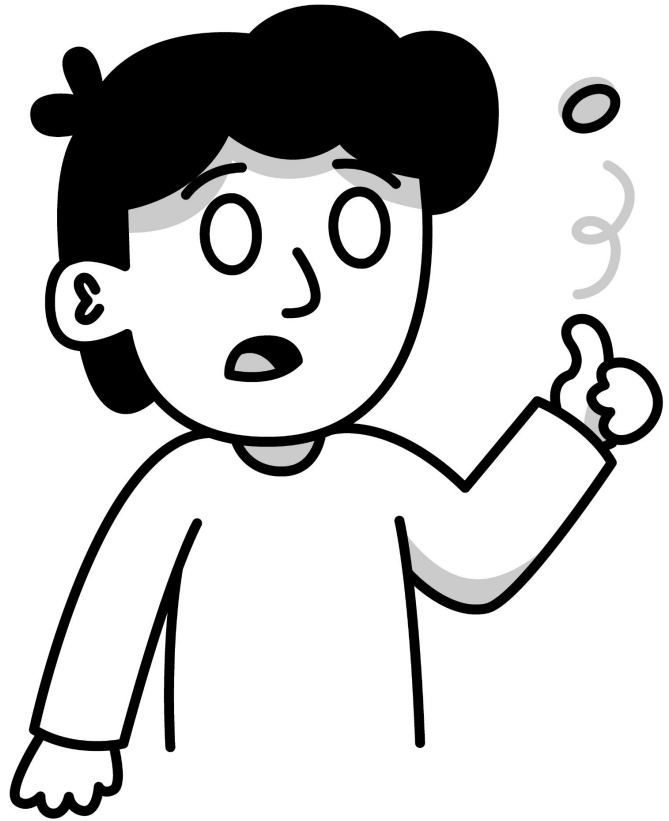


Let’s talk about **sampling**

Head Sampling: Easy, kind of bad

Tail Sampling: Incredibly difficult to get right, but *sometimes* worth the pay-off





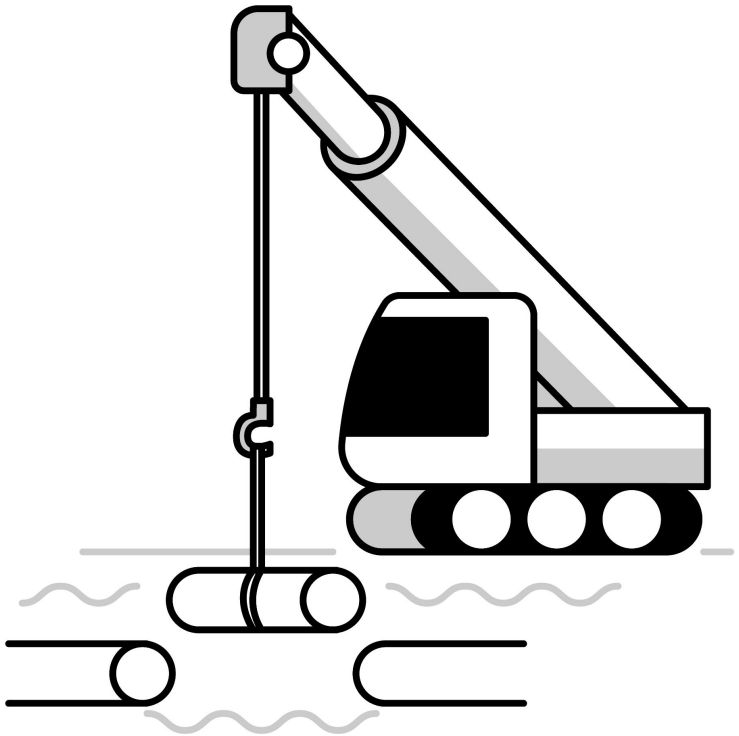
Head sampling is a no brainer, but ***include the sampling rate in your spans*** so you can properly recreate your metrics

Tail Sampling requires:

- **A collector**, with buffering
- **Load balancing** for that collector to get the right spans to the right place
- **Resource Considerations** for how long to buffer for







You’re going to need a ***collector***: Something that can take telemetry, transform it, and export it

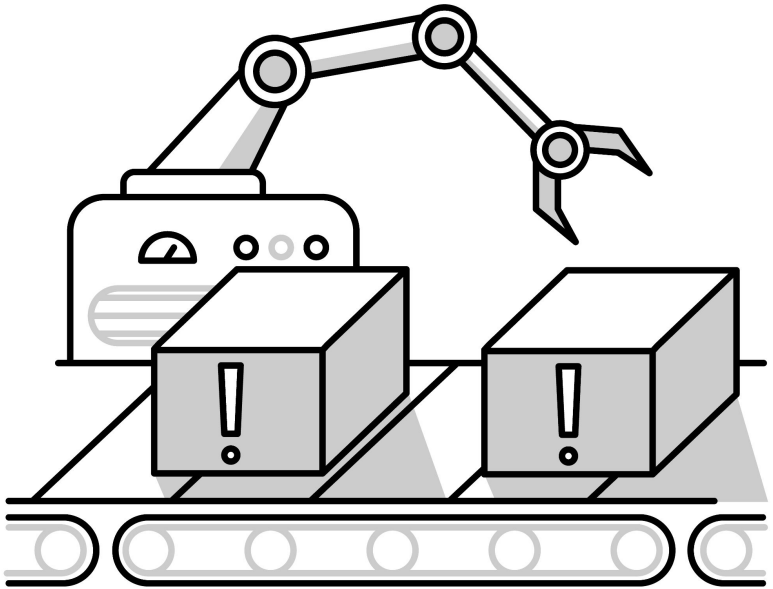
- *Ideally* something that can transform one telemetry type into another

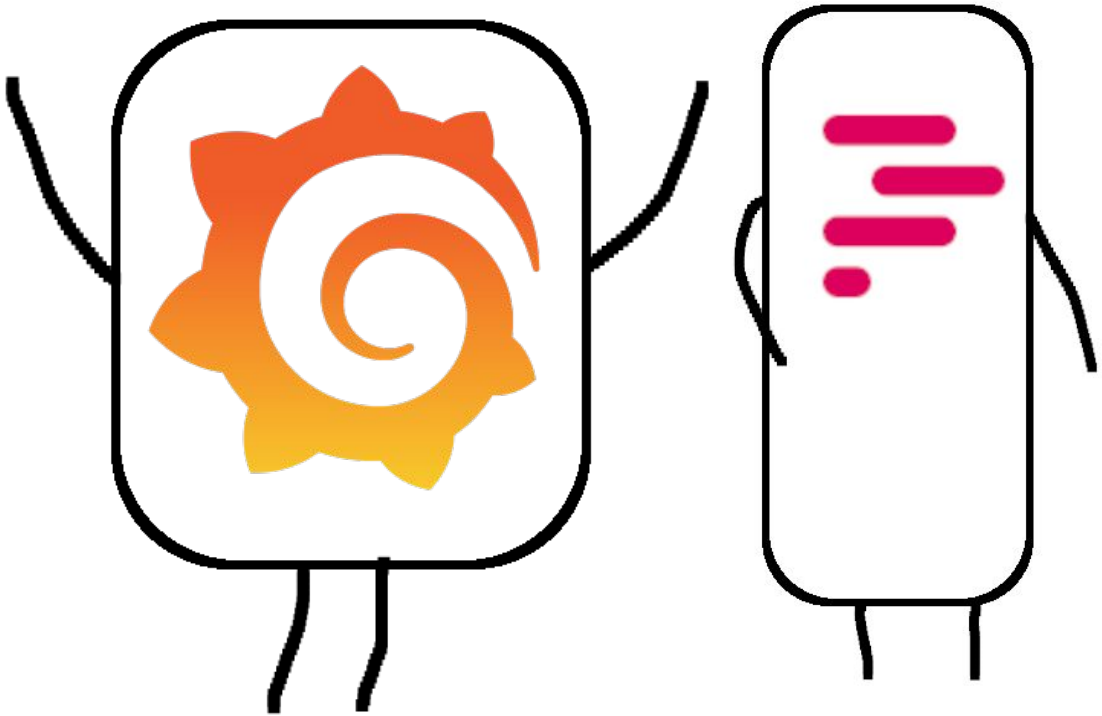


Again, lots of options

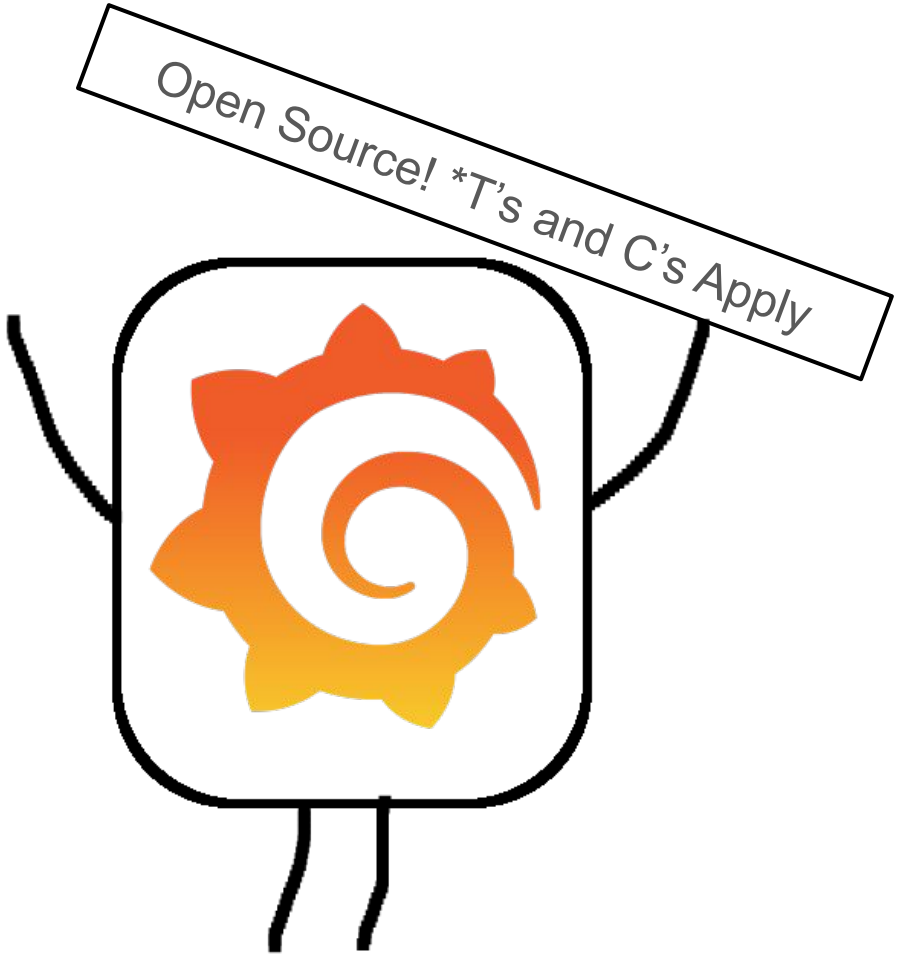
- The OTel Collector
- Grafana Alloy, which is just the OTel Collector
- Vector
- Telegraf

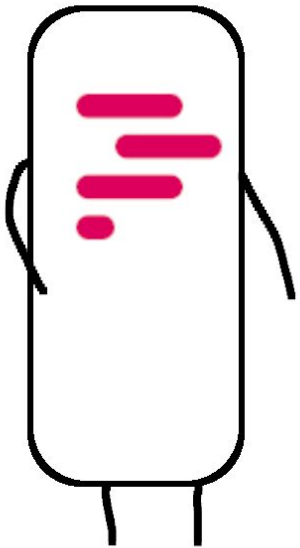
We need something to *tie everything together*





- Grafana locks a lot of its more useful features behind paying them money
- Even when they're implemented as Open Source versions, they get left behind
- Capitalism ruins everything, even open source projects





Perses has:

- A better API for automated dashboards
- Better Open Source support
- **Ephemeral Dashboards**

It’s interesting to make the distinction between **business dashboards** and **debugging dashboards**

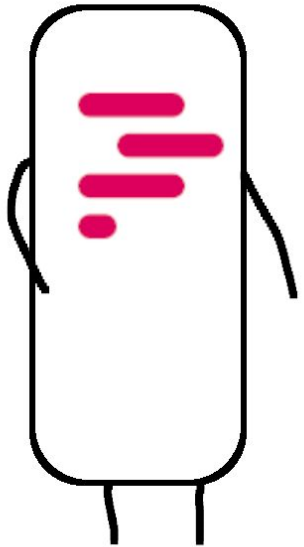
Business dashboards: Long lived, monitored often, keep the business going

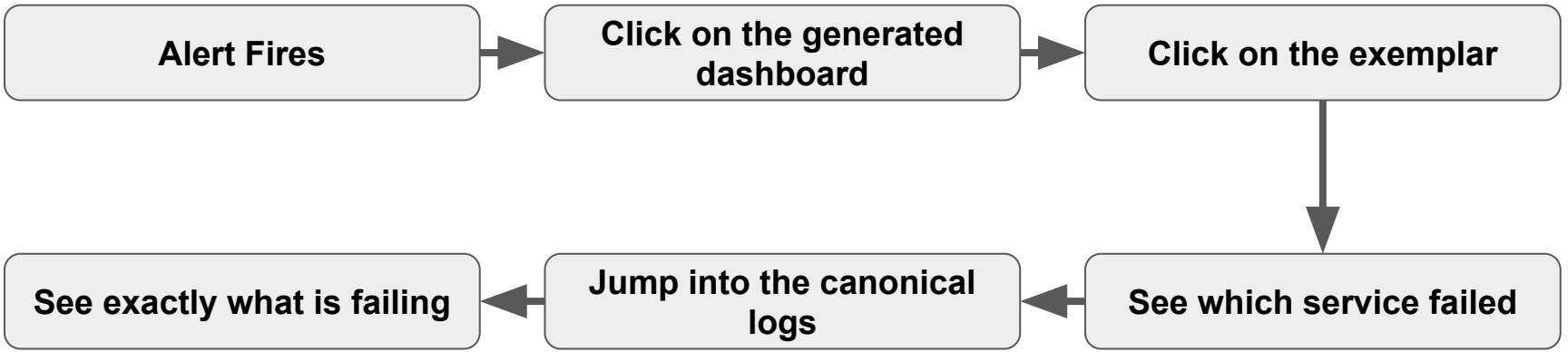
It’s interesting to make the distinction between **business dashboards** and **debugging dashboards**

Business dashboards: Long lived, monitored often, keep the business going

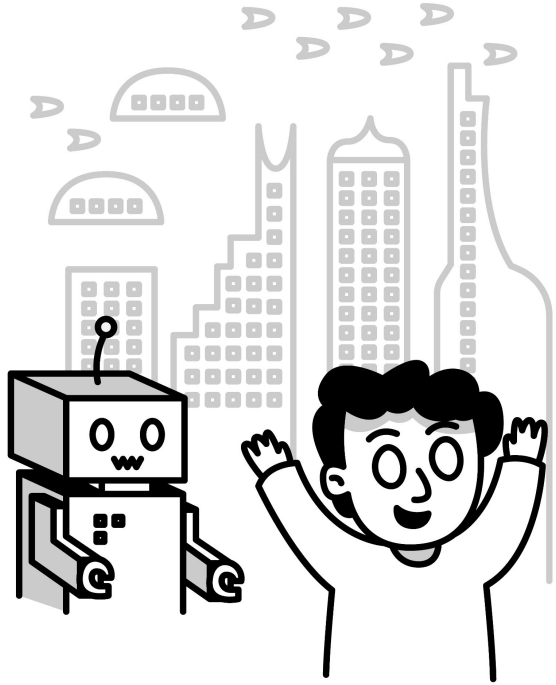
Debugging Dashboards: Created adhoc, used once, but *tend to stick around*

Perses has **ephemeral dashboards** to automate that cleanup for you, and has a nicer API so that you can *generate* those dashboards on the fly




Alert Fires
Click on the generated dashboard
Click on the exemplar
See which service failed
Jump into the canonical logs
See exactly what is failing

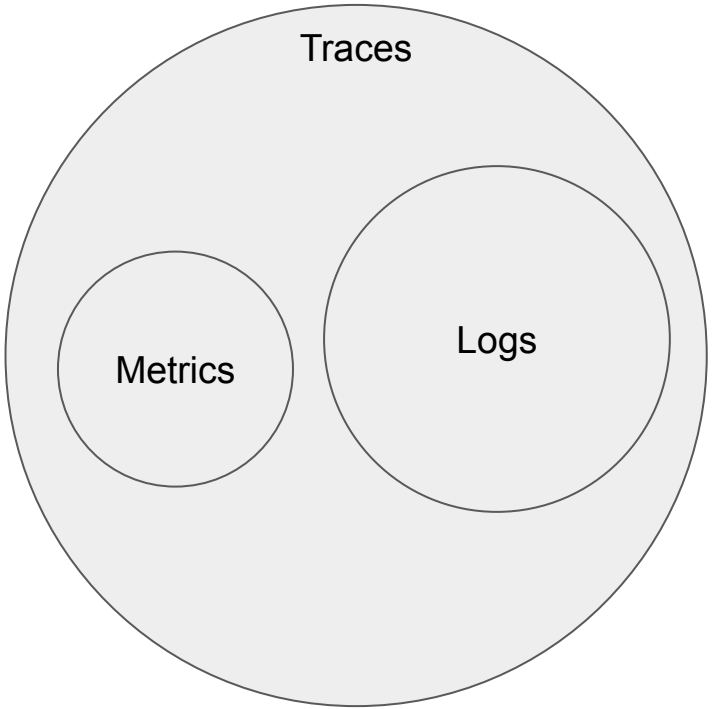
But what about the future?

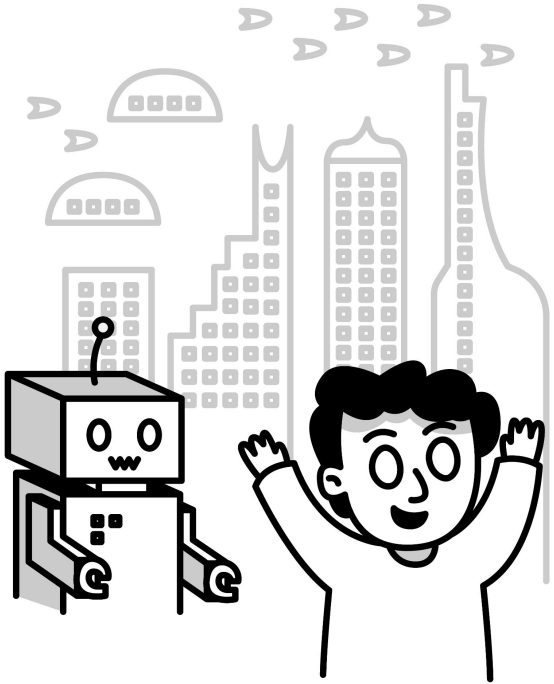


When you have your three pillars, there's **a lot of duplication**

Your **logs** are just subsets of your **traces**

Your **metrics** are just aggregations over the same data



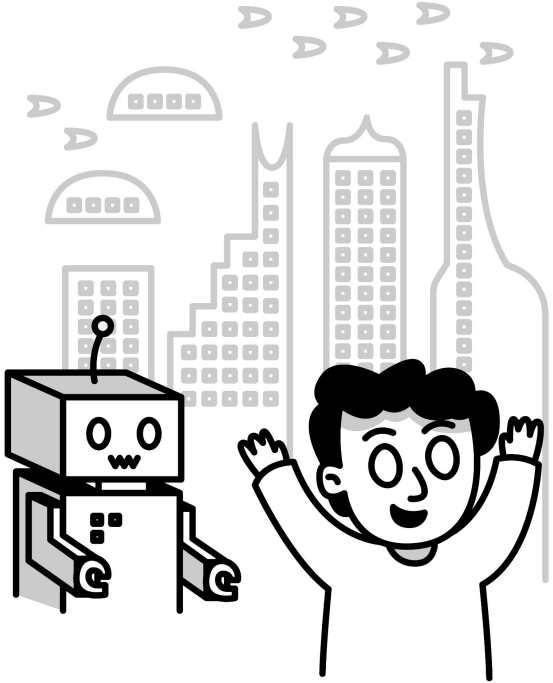


We’re starting to see this:

- Things like SigNoz, and HyperDX are trying to do everything in one package
- Pipelining tools are developing connectors
- Tracing tools are developing their own metrics generation and logs support

A future where:

- Everything flows out of your wide, high cardinality traces
- The three pillars still exist, but are tied together
- Everything just flows from one system to another



Thanks!

Slides

Art by Alicia Edwards
https://www.aliciaillustration.com/

@colindou.ch on BlueSky